CMAN Codefest 5 — Weight-Stationary Systolic Array Trace

2×2 array computing C = A × B with weights of B preloaded into the PEs

Problem	Compute C = A × B on a 2×2 weight-stationary systolic array.
Inputs	`A = [[1, 2], [3, 4]],  B = [[5, 6], [7, 8]]`
Expected	`C = [[19, 22], [43, 50]]`

Task 1 — PE Array with Preloaded Weights

Each PE is preloaded with the corresponding element of B and holds it fixed for the entire computation:

`PE[0][0] = 5`	`PE[0][1] = 6`
`PE[1][0] = 7`	`PE[1][1] = 8`

PE[i][j] = B[i][j]. Row 0 of A streams into row 0 of the array; row 1 of A streams into row 1 of the array. Each cycle, both rows feed an input value into every PE in their row.

Task 2 — Cycle-by-Cycle Trace

Each cycle, the two rows of A send one value each into every PE in their row of the array. Each PE computes *previous + (input × weight)*, and we record the running PE values as a 2×2 snapshot C_k after each cycle. The final result emerges in cycle 4.

Cyc	Row 0 sends	Row 1 sends	PE[0][0] (w=5)	PE[0][1] (w=6)	PE[1][0] (w=7)	PE[1][1] (w=8)	Snapshot
1	**1**	**3**	`0 + (1×5) = 5`	`0 + (1×6) = 6`	`0 + (3×7) = 21`	`0 + (3×8) = 24`	C_1
2	**2**	**4**	`5 + (2×5) = 15`	`6 + (2×6) = 18`	`21 + (4×7) = 49`	`24 + (4×8) = 56`	C_2
3	**1**	**3**	`0 + (1×5) = 5`	`0 + (1×6) = 6`	`0 + (3×5) = 15`	`0 + (3×6) = 18`	C_3
4	**2**	**4**	`5 + (2×7) = 19`	`6 + (2×8) = 22`	`15 + (4×7) = 43`	`18 + (4×8) = 50`	C_4 = C

Snapshot matrices after each cycle

After each cycle, the values currently held in the four PEs form a 2×2 matrix. The snapshot after cycle 4 is the final output C.

After Cycle 1 (C_1)

5	6
21	24

After Cycle 2 (C_2)

15	18
49	56

After Cycle 3 (C_3)

5	6
15	18

After Cycle 4 — Final C

19	22
43	50

Final result: C = [[19, 22], [43, 50]] ✓ matches the expected output.

Task 3 — MAC Count, Reuse, and Memory Accesses

(a) Total MAC operations: 8 total MAC operations performed.

(b) Input reuse: every number was used 2 times.

Operand	Where reused	Reuse count
`Each A value`	Multiplied against both PE weights in its row	**2**
`Each B weight`	Multiplied against 2 different A values over the run	**2**

(c) Off-chip memory accesses: There was one access for each value in A which gives 4 accesses, one access for each weight in B which gives another 4 accesses, and one access for each answer written into C which gives the last 4 accesses. There are 12 memory accesses in total.

Tensor	Direction	Accesses
A	Read — one access for each value in A	**4**
B	Read — one access for each weight in B (preloaded once)	**4**
C	Write — one access for each answer written into C	**4**
Total		**12**

Task 4 — Output-Stationary Variant

In weight-stationary, the weights of B are preloaded into the PEs and stay fixed throughout the computation. **In output-stationary, the partial sums of C stay fixed inside the PEs and accumulate, while both A values and B values stream in from memory every cycle.**